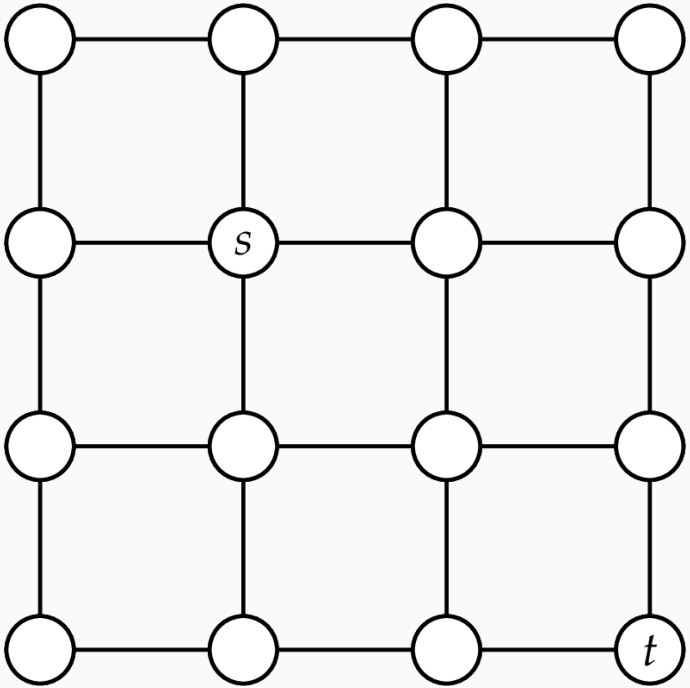
# Shortest Paths

Prof. Timothy Sun

# Unweighted shortest paths

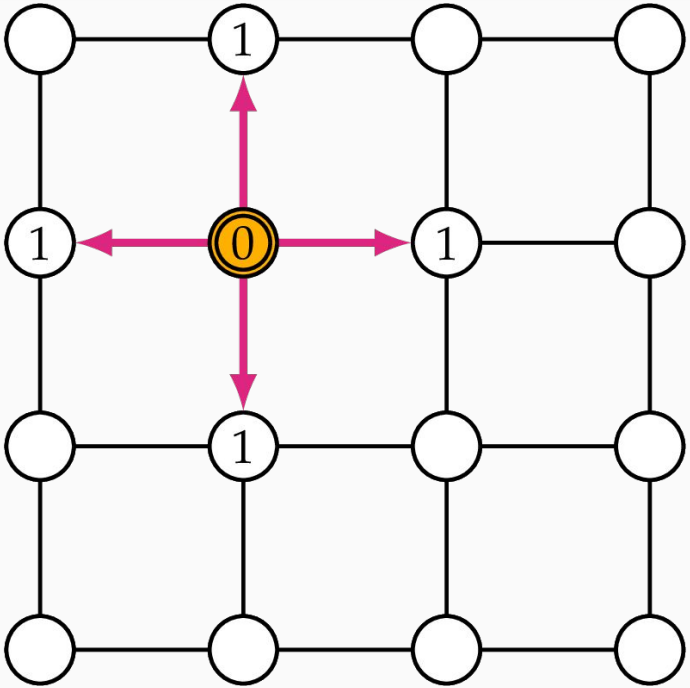
Given an ***unweighted*** graph, how do you find the shortest path between two vertices?



# Unweighted shortest paths

Given an ***unweighted*** graph, how do you find the shortest path between two vertices?

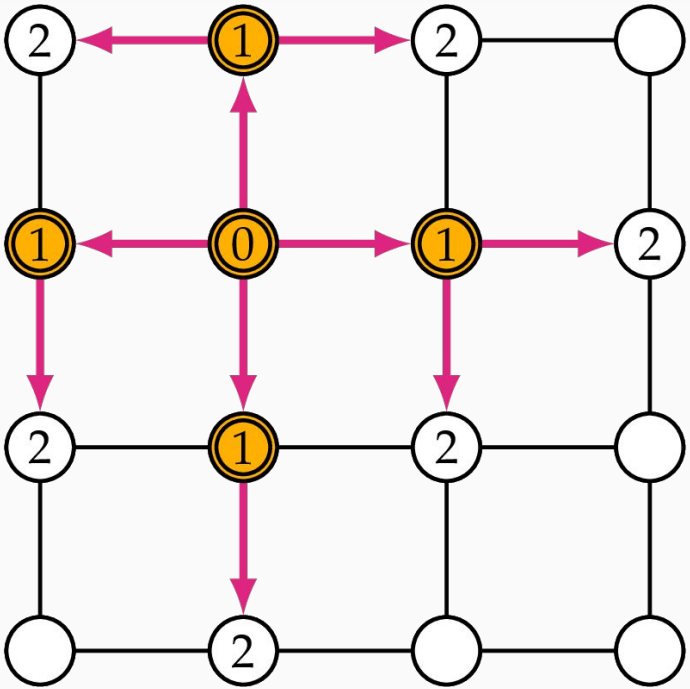
**Breadth-first search**



# Unweighted shortest paths

Given an ***unweighted*** graph, how do you find the shortest path between two vertices?

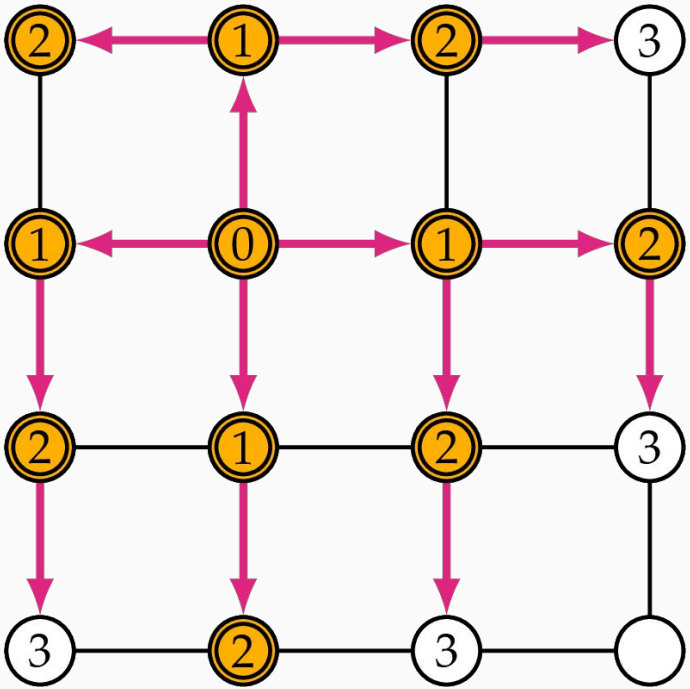
**Breadth-first search**



# Unweighted shortest paths

Given an ***unweighted*** graph, how do you find the shortest path between two vertices?

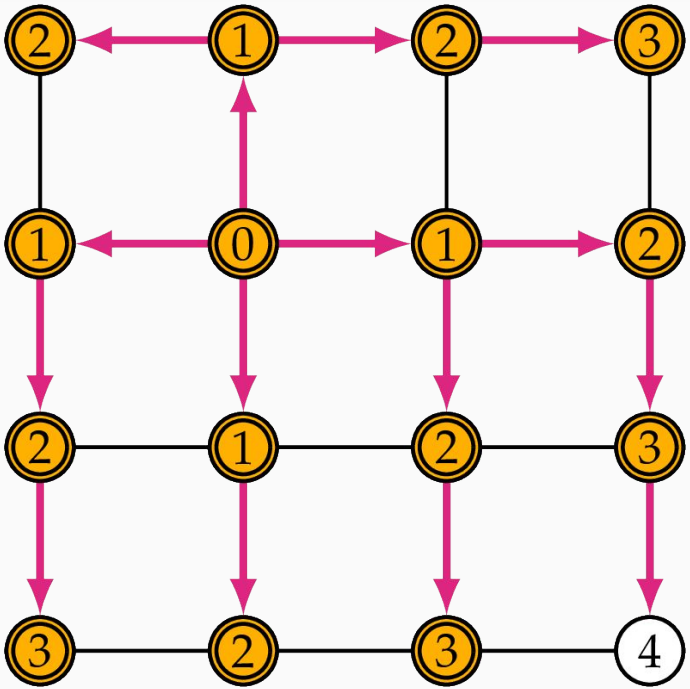
**Breadth-first search**



# Unweighted shortest paths

Given an ***unweighted*** graph, how do you find the shortest path between two vertices?

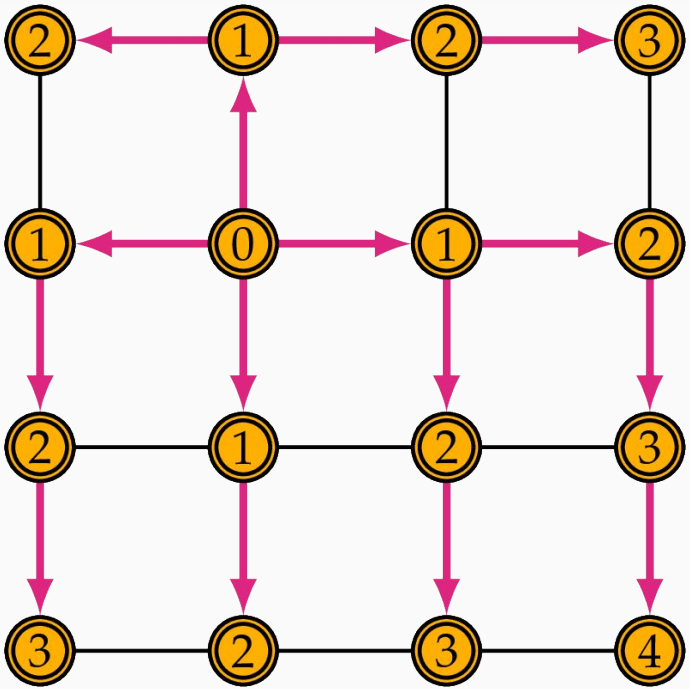
**Breadth-first search**



# Unweighted shortest paths

Given an ***unweighted*** graph, how do you find the shortest path between two vertices?

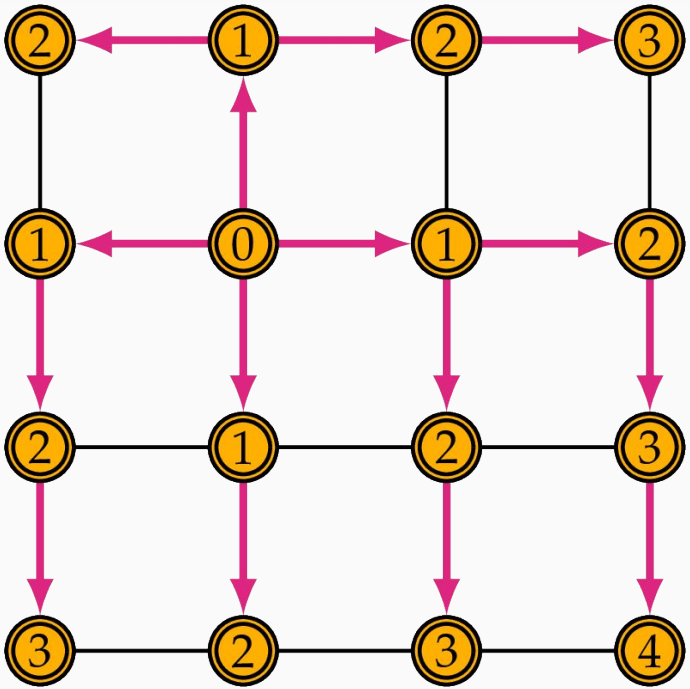
**Breadth-first search**



# The problem

Most shortest path algorithms solve a more general problem:

**Single-Source Shortest Paths (SSSP)**

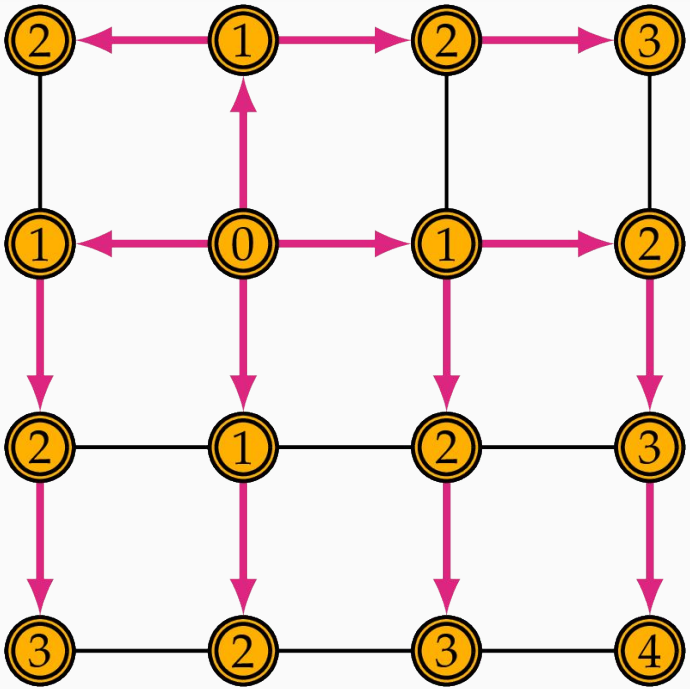


# The problem

Most shortest path algorithms solve a more general problem:

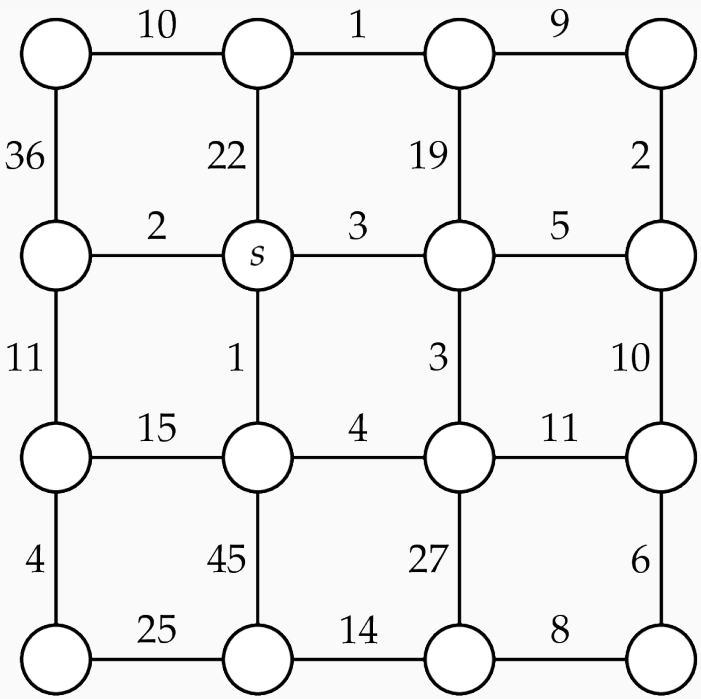
**Single-Source Shortest Paths (SSSP)**

Output: ***spanning tree*** rooted at starting vertex, ***vertex distances***.



# Weighted shortest paths

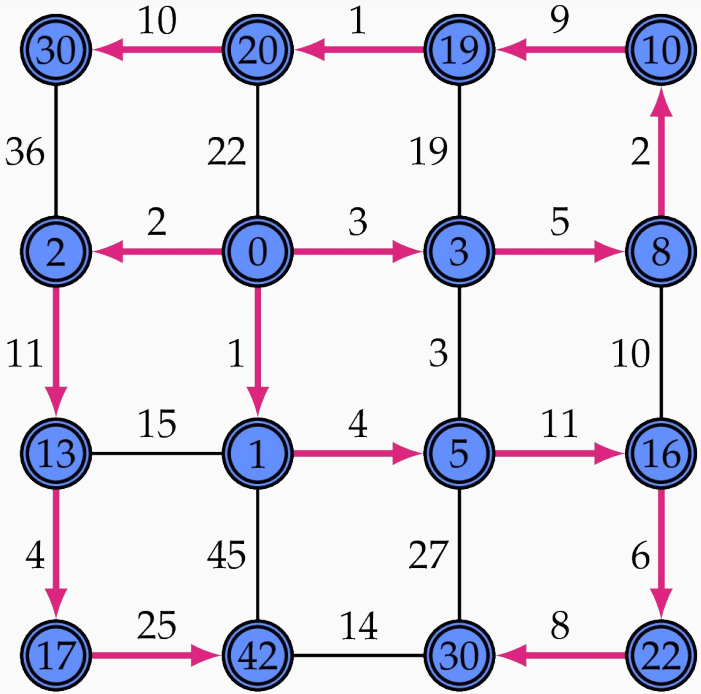
What about the ***weighted*** version?



# Weighted shortest paths

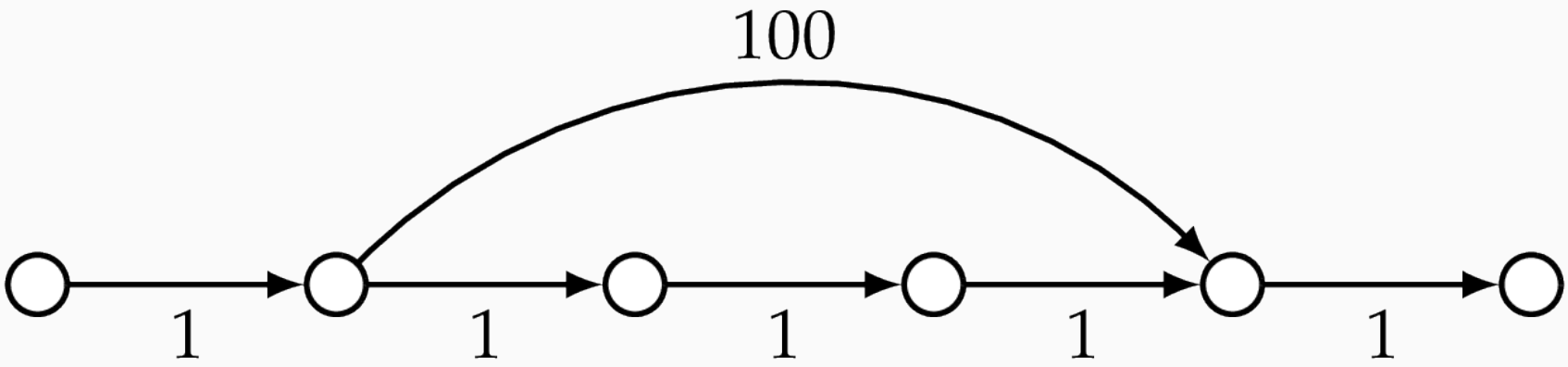
What about the ***weighted*** version?

SSSP tree can be very different from BFS tree



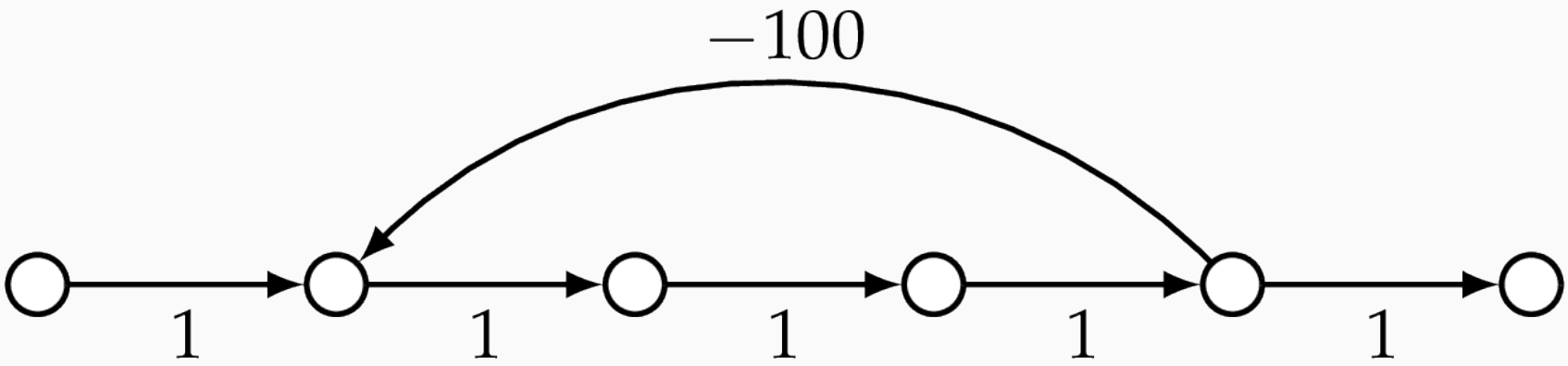
# Weighted shortest paths

A shortest path may use many short edges.



# Weighted shortest paths

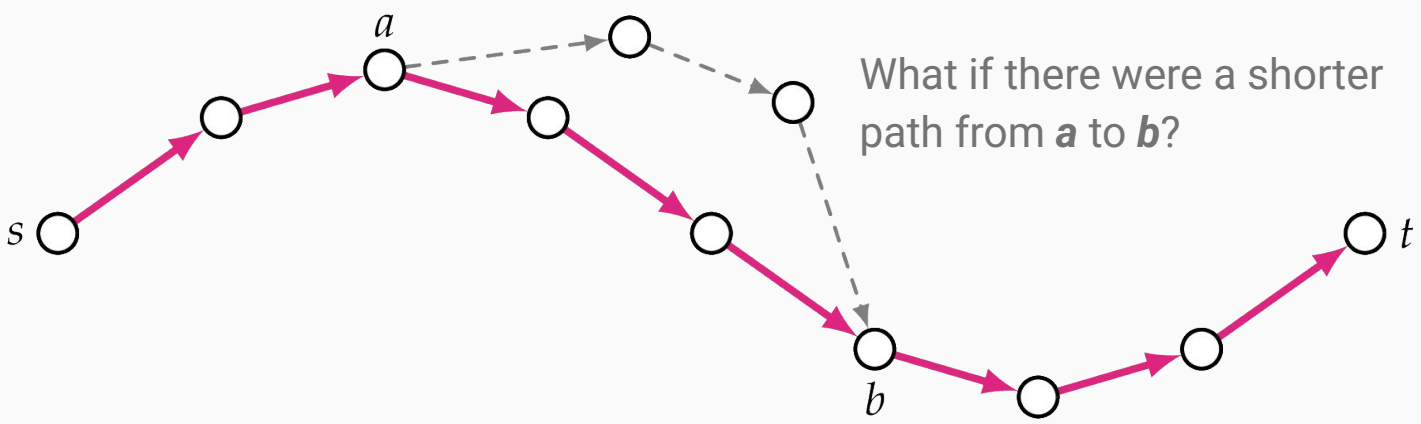
**Assumption**: positive edge weights




What's the "shortest path" from left to right in this graph?

# Subpath optimality

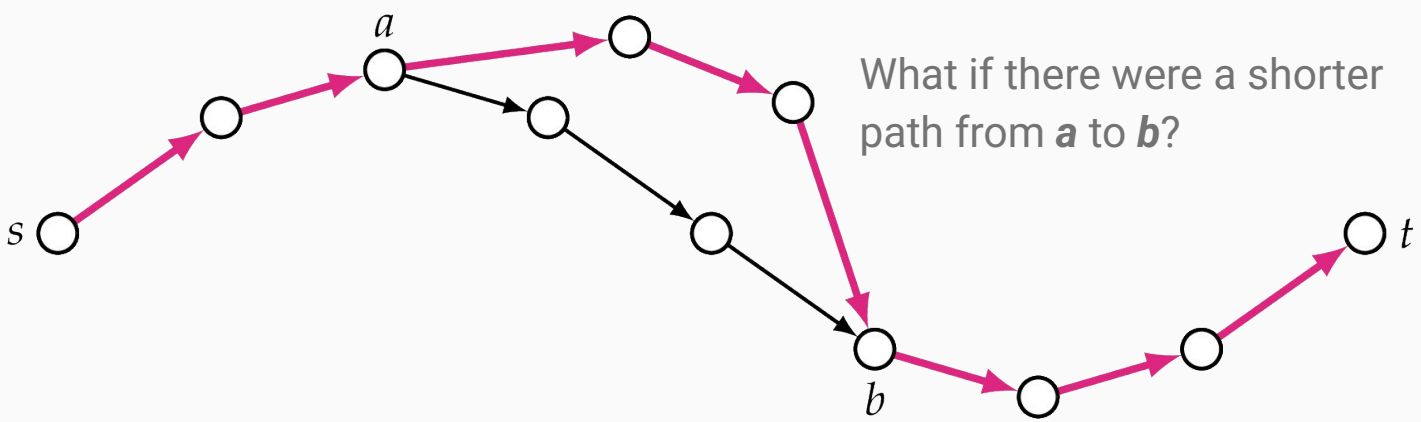
If we have a shortest path from ***s*** to ***t***, all “subpaths” are shortest as well.




What if there were a shorter path from ***a*** to ***b***?

# Subpath optimality

If we have a shortest path from ***s*** to ***t***, all “subpaths” are shortest as well.




What if there were a shorter path from ***a*** to ***b***?

The same shortcut could be used for a path from ***s*** to ***t***.

# Generic setup

On input ***G*** and start vertex ***s***:

# Generic setup

On input ***G*** and start vertex ***s***:

Each vertex ***v*** has:

- dist(***v***): length of best path from ***s*** to ***v*** so far
- pred(***v***): predecessor in best path so far

# Generic setup

On input ***G*** and start vertex ***s***:

Each vertex ***v*** has:

- dist(***v***): length of best path from ***s*** to ***v*** so far
- pred(***v***): predecessor in best path so far

At the end, hopefully dist(***v***) is the actual shortest path

# Initialization

Before we've explored the graph, what's the best paths found so far?

# Initialization

Before we've explored the graph, what's the best paths found so far?

**Haven't found anything yet…**

How should we initialize dist( )?

# Initialization

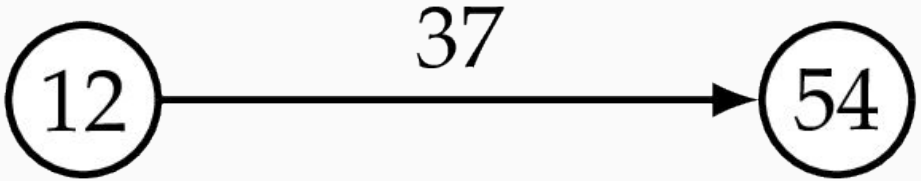
Before we've explored the graph, what's the best paths found so far?

**Haven't found anything yet...**

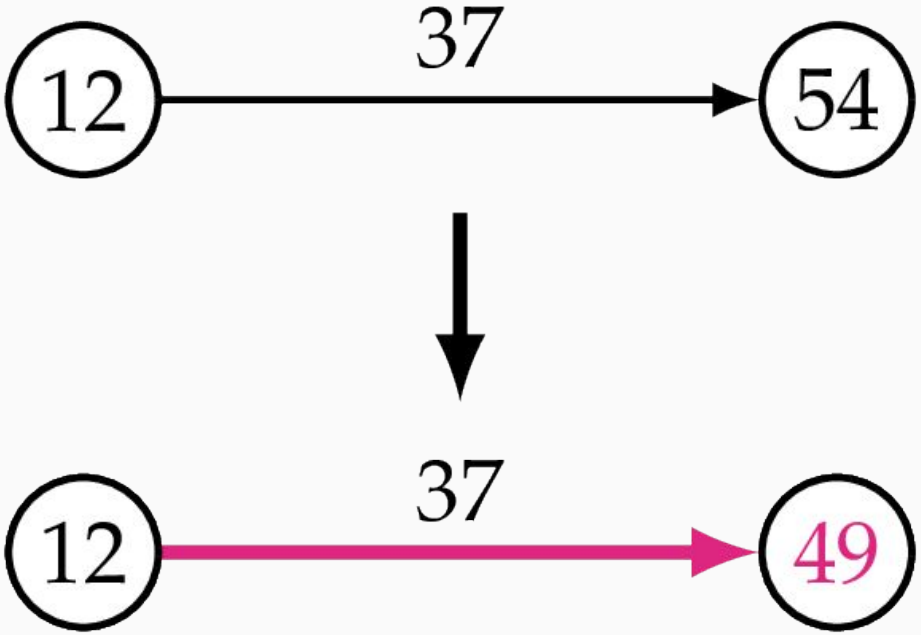
How should we initialize dist( )?

- dist(***s***) = 0
- dist(***v***) = ∞, for all other ***v***

# Relaxing “tense” edges



# Relaxing “tense” edges



# Edge relaxation

If it's tense, relax it:

$$
\begin{aligned}
&\textsc{Relax}(u \to v):\\
&\qquad \textbf{if } dist(u) + w(u \to v) < dist(v)\\
&\qquad\qquad dist(v) := dist(u) + w(u \to v)\\
&\qquad\qquad pred(v) := u
\end{aligned}
$$

# Edge relaxation

Algorithm idea:

- Keep relaxing until no more relaxations are possible

# Edge relaxation

Algorithm idea:

- Keep relaxing until no more relaxations are possible

***If*** this terminates, we have a SSSP tree (induction, subpath optimality)

# Edge relaxation

Algorithm idea:

- Keep relaxing until no more relaxations are possible

***If*** this terminates, we have a SSSP tree (induction, subpath optimality)

- dist(***v***) = true shortest distance from ***s*** to ***v***

# Edge relaxation

Algorithm idea:

- Keep relaxing until no more relaxations are possible

1. **How many relaxations?**
2. **Where to quickly find a tense edge?**

# Edge relaxation - BFS

For unweighted graphs and BFS, what is a relaxation?

$$
\begin{aligned}
&\textsc{Relax}(u \to v):\\
&\qquad \textbf{if } dist(u) + w(u \to v) < dist(v)\\
&\qquad\qquad dist(v) := dist(u) + w(u \to v)\\
&\qquad\qquad pred(v) := u
\end{aligned}
$$

# Edge relaxation - BFS

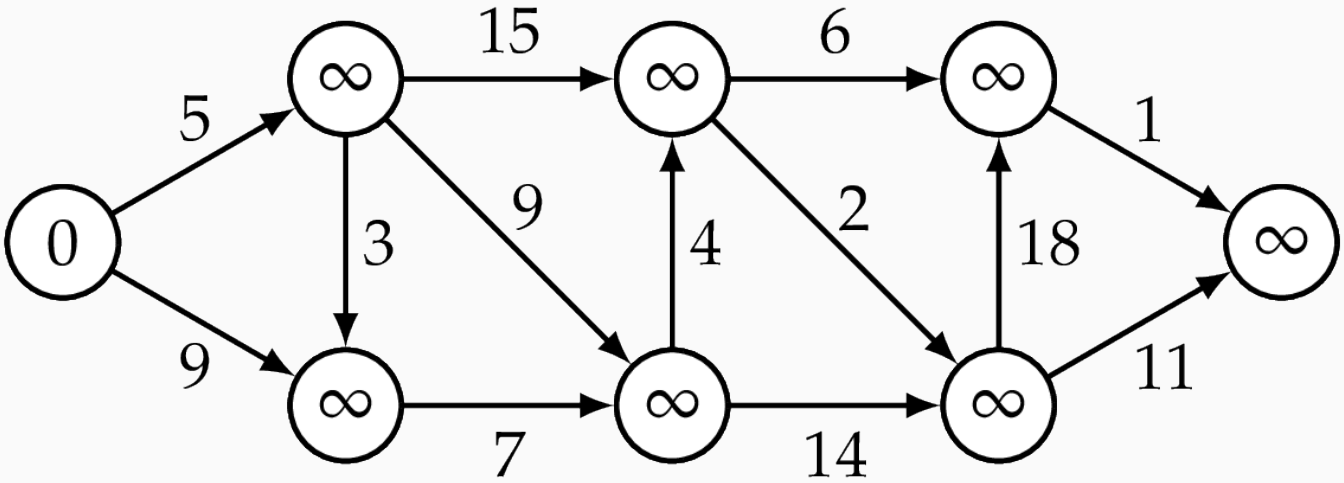
For unweighted graphs and BFS, what is a relaxation?

$$
\begin{aligned}
&\textsc{Relax}(u \to v):\\
&\qquad \textbf{if } dist(u) + w(u \to v) < dist(v)\\
&\qquad\qquad dist(v) := dist(u) + w(u \to v)\\
&\qquad\qquad pred(v) := u
\end{aligned}
$$

***u*** = vertex in queue, ***v*** = unvisited vertex

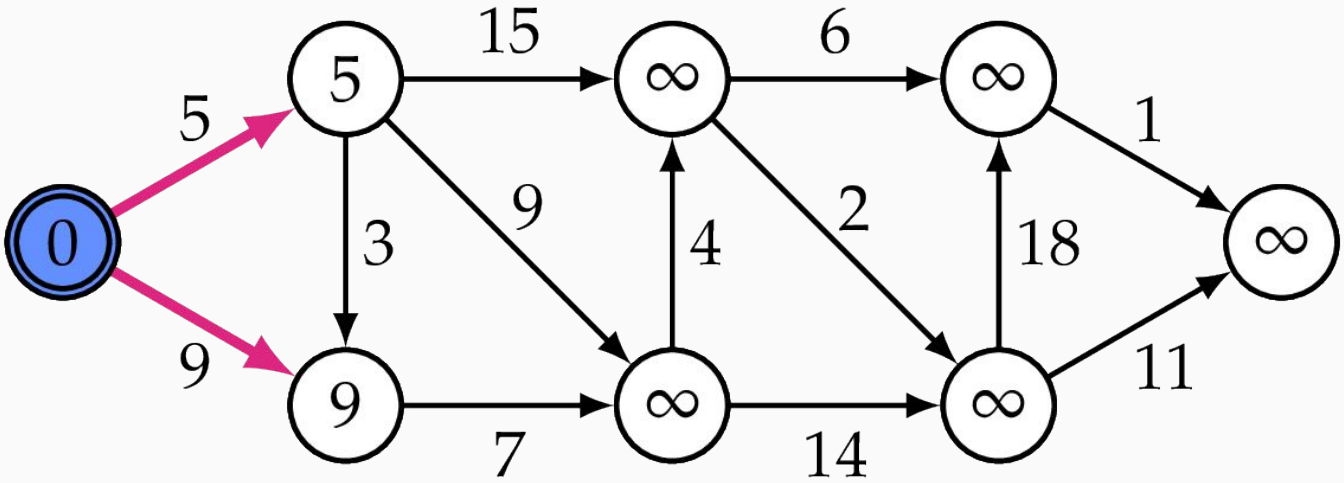
# Dijkstra's algorithm

"Of all unvisited vertices, visit ***v*** with smallest dist(***v***), and try to relax its edges."



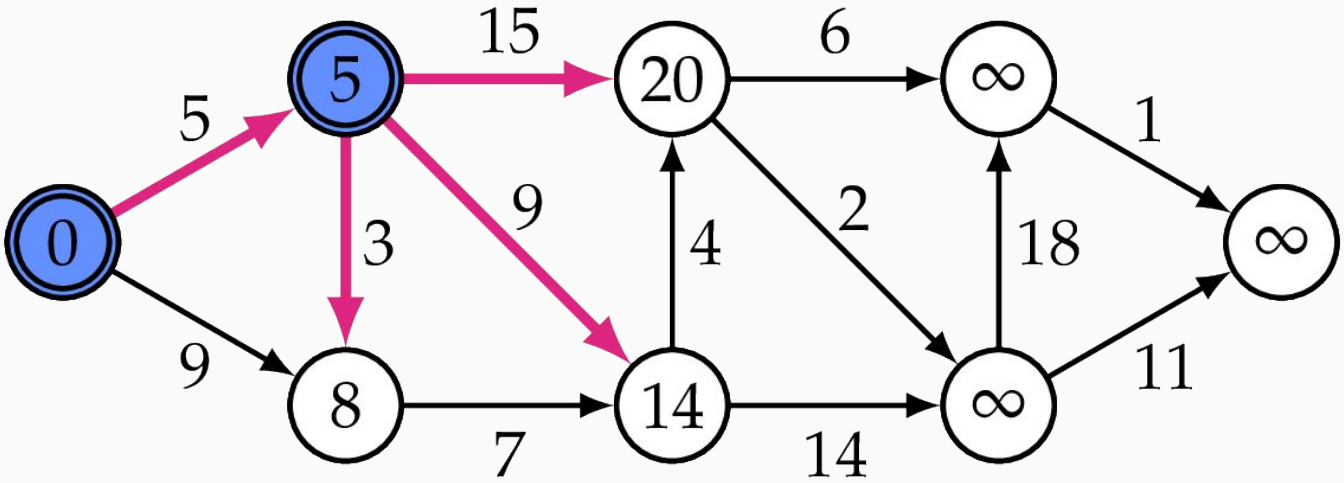
# Dijkstra's algorithm

"Of all unvisited vertices, visit ***v*** with smallest dist(***v***), and try to relax its edges."



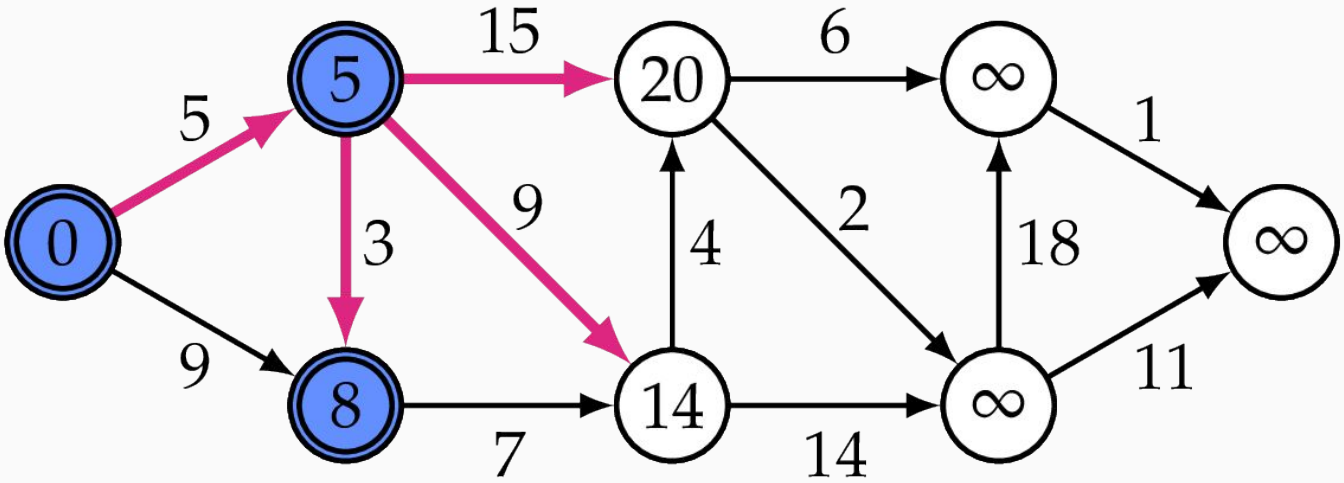
# Dijkstra's algorithm

"Of all unvisited vertices, visit ***v*** with smallest dist(***v***), and try to relax its edges."



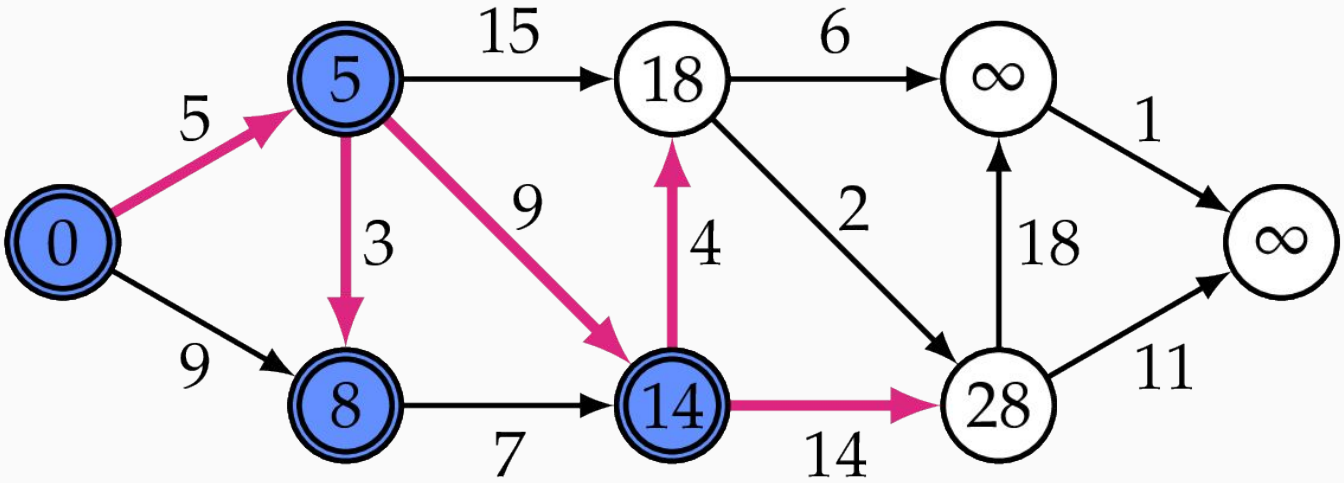
# Dijkstra’s algorithm

“Of all unvisited vertices, visit ***v*** with smallest dist(***v***), and try to relax its edges.”



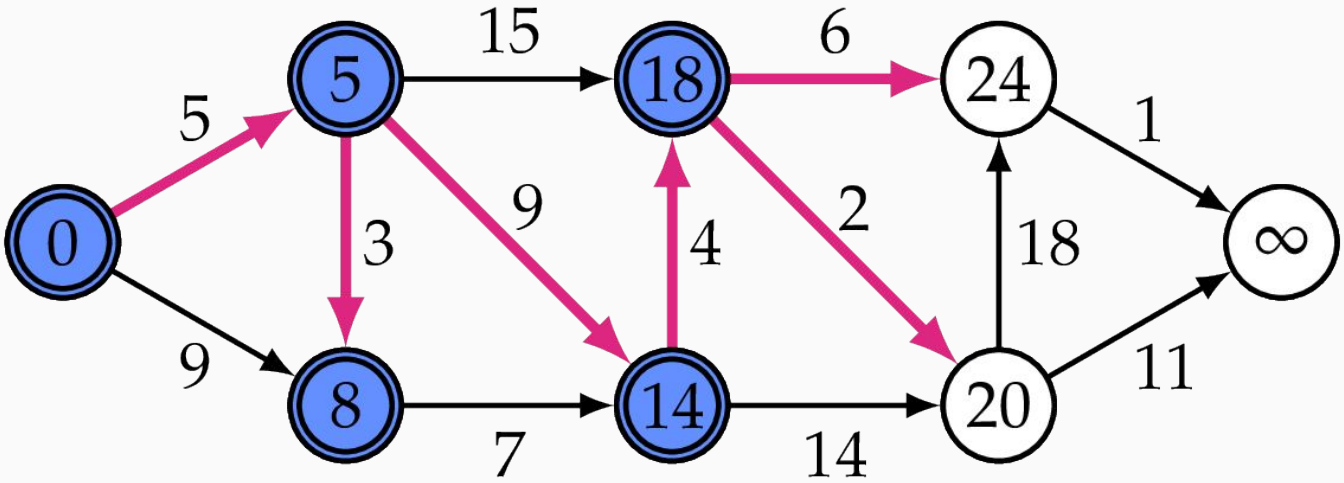
# Dijkstra's algorithm

"Of all unvisited vertices, visit ***v*** with smallest dist(***v***), and try to relax its edges."



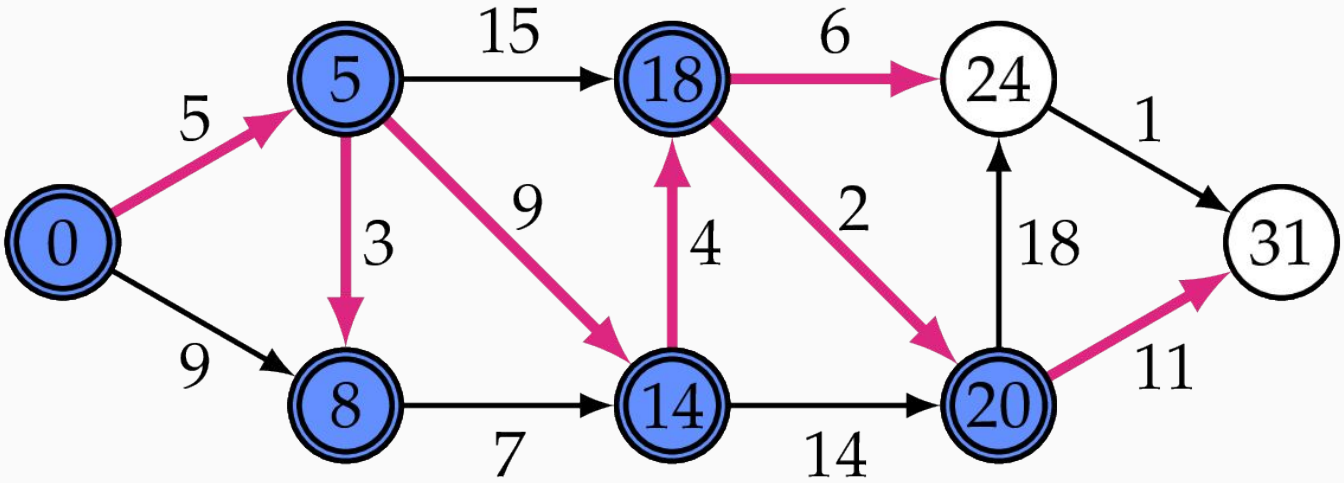
# Dijkstra's algorithm

"Of all unvisited vertices, visit ***v*** with smallest dist(***v***), and try to relax its edges."



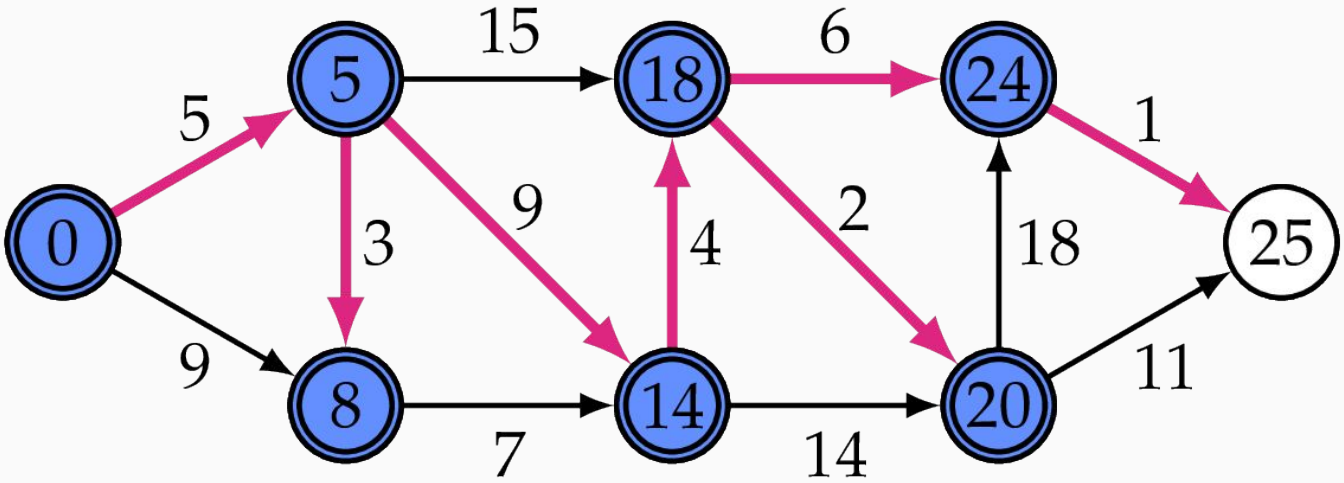
# Dijkstra’s algorithm

“Of all unvisited vertices, visit ***v*** with smallest dist(***v***), and try to relax its edges.”



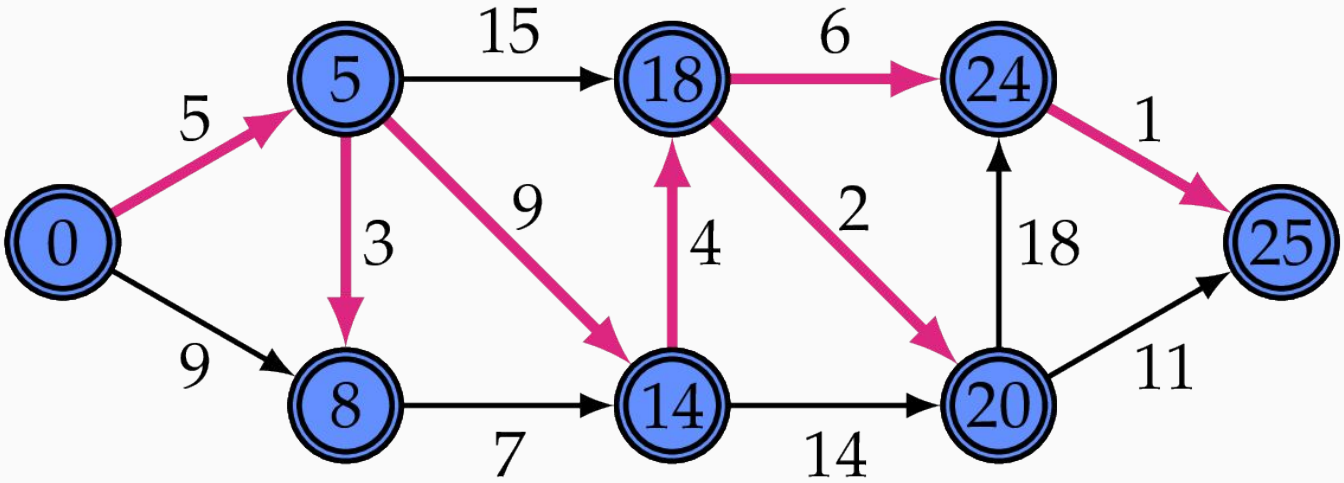
# Dijkstra's algorithm

"Of all unvisited vertices, visit ***v*** with smallest dist(***v***), and try to relax its edges."



# Dijkstra's algorithm

"Of all unvisited vertices, visit ***v*** with smallest dist(***v***), and try to relax its edges."



# Dijkstra's algorithm

"Of all unvisited vertices, visit ***v*** with smallest dist(***v***), and try to relax its edges."

```
DIJKSTRA(G(V, E), s):
    dist(s) := 0
    for v ∈ V \ {s}
        dist(v) := ∞
    PQ := PRIORITYQUEUE()
    for v ∈ V
        PQ.INSERT(v, dist(v))
    while |PQ| > 0
        u := PQ.EXTRACTMIN()
        for each edge u → v
            if u → v is tense
                RELAX(u → v)
                PQ.DECREASEKEY(v, dist(v))
```

# Dijkstra's algorithm

"Of all unvisited vertices, visit ***v*** with smallest dist(***v***), and try to relax its edges."

What's the runtime?

```
DIJKSTRA(G(V, E), s):
    dist(s) := 0
    for v ∈ V \ {s}
        dist(v) := ∞
    PQ := PRIORITYQUEUE()
    for v ∈ V
        PQ.INSERT(v, dist(v))
    while |PQ| > 0
        u := PQ.EXTRACTMIN()
        for each edge u → v
            if u → v is tense
                RELAX(u → v)
                PQ.DECREASEKEY(v, dist(v))
```

# Dijkstra's algorithm

"Of all unvisited vertices, visit ***v*** with smallest dist(***v***), and try to relax its edges."

What's the runtime?

**Depends on the priority queue!**

```
DIJKSTRA(G(V, E), s):
    dist(s) := 0
    for v ∈ V \ {s}
        dist(v) := ∞
    PQ := PRIORITYQUEUE()
    for v ∈ V
        PQ.INSERT(v, dist(v))
    while |PQ| > 0
        u := PQ.EXTRACTMIN()
        for each edge u → v
            if u → v is tense
                RELAX(u → v)
                PQ.DECREASEKEY(v, dist(v))
```

# Dijkstra's algorithm

Binary heap:

- ExtractMin: O(|***V***| log |***V***|)
- DecreaseKey: O(|***E***| log |***V***|)

Total: O(|***E***| log |***V***|)

```
DIJKSTRA(G(V, E), s):
    dist(s) := 0
    for v ∈ V \ {s}
        dist(v) := ∞
    PQ := PRIORITYQUEUE()
    for v ∈ V
        PQ.INSERT(v, dist(v))
    while |PQ| > 0
        u := PQ.EXTRACTMIN()
        for each edge u → v
            if u → v is tense
                RELAX(u → v)
                PQ.DECREASEKEY(v, dist(v))
```

# Dijkstra's algorithm

Unsorted array(!):

- ExtractMin: $O(|\boldsymbol{V}|^2)$
- DecreaseKey: $O(|\boldsymbol{E}|)$

Total: $O(|\boldsymbol{V}|^2)$

```
DIJKSTRA(G(V, E), s):
    dist(s) := 0
    for v ∈ V \ {s}
        dist(v) := ∞
    PQ := PRIORITYQUEUE()
    for v ∈ V
        PQ.INSERT(v, dist(v))
    while |PQ| > 0
        u := PQ.EXTRACTMIN()
        for each edge u → v
            if u → v is tense
                RELAX(u → v)
                PQ.DECREASEKEY(v, dist(v))
```

# Dijkstra's algorithm

Which is better,

$O(|\boldsymbol{E}| \log |\boldsymbol{V}|)$ or $O(|\boldsymbol{V}|^2)$?

```
DIJKSTRA(G(V, E), s):
    dist(s) := 0
    for v ∈ V \ {s}
        dist(v) := ∞
    PQ := PRIORITYQUEUE()
    for v ∈ V
        PQ.INSERT(v, dist(v))
    while |PQ| > 0
        u := PQ.EXTRACTMIN()
        for each edge u → v
            if u → v is tense
                RELAX(u → v)
                PQ.DECREASEKEY(v, dist(v))
```

# Dijkstra's algorithm

Which is better,

O(|***E***| log |***V***|) or O(|***V***|$^2$)?

**Depends on edge density.**

```
DIJKSTRA(G(V, E), s):
    dist(s) := 0
    for v ∈ V \ {s}
        dist(v) := ∞
    PQ := PRIORITYQUEUE()
    for v ∈ V
        PQ.INSERT(v, dist(v))
    while |PQ| > 0
        u := PQ.EXTRACTMIN()
        for each edge u → v
            if u → v is tense
                RELAX(u → v)
                PQ.DECREASEKEY(v, dist(v))
```

# Edge relaxation

**Greedy algorithm:** why is ***u*** the best place to grow the SSSP tree?

```
DIJKSTRA(G(V, E), s):
    dist(s) := 0
    for v ∈ V \ {s}
        dist(v) := ∞
    PQ := PRIORITYQUEUE()
    for v ∈ V
        PQ.INSERT(v, dist(v))
    while |PQ| > 0
        u := PQ.EXTRACTMIN()
        for each edge u → v
            if u → v is tense
                RELAX(u → v)
                PQ.DECREASEKEY(v, dist(v))
```

# Edge relaxation

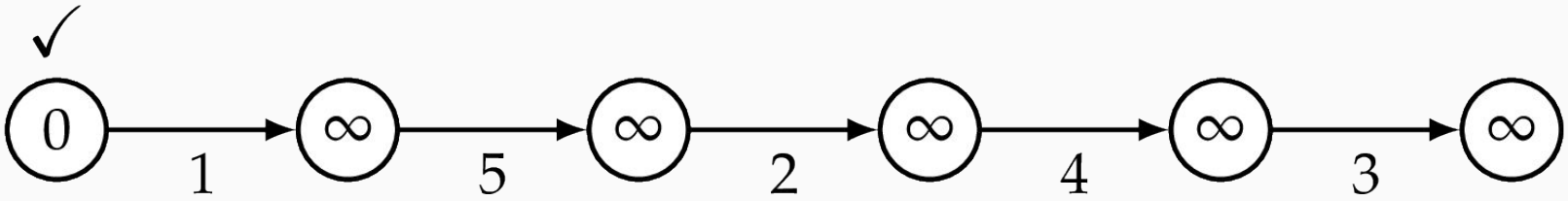
**Greedy algorithm:** why is ***u*** the best place to grow the SSSP tree?

Claim: Once ***u*** is popped, shortest path to ***u*** found: dist(***u***) = true distance

```
DIJKSTRA(G(V, E), s):
    dist(s) := 0
    for v ∈ V \ {s}
        dist(v) := ∞
    PQ := PRIORITYQUEUE()
    for v ∈ V
        PQ.INSERT(v, dist(v))
    while |PQ| > 0
        u := PQ.EXTRACTMIN()
        for each edge u → v
            if u → v is tense
                RELAX(u → v)
                PQ.DECREASEKEY(v, dist(v))
```

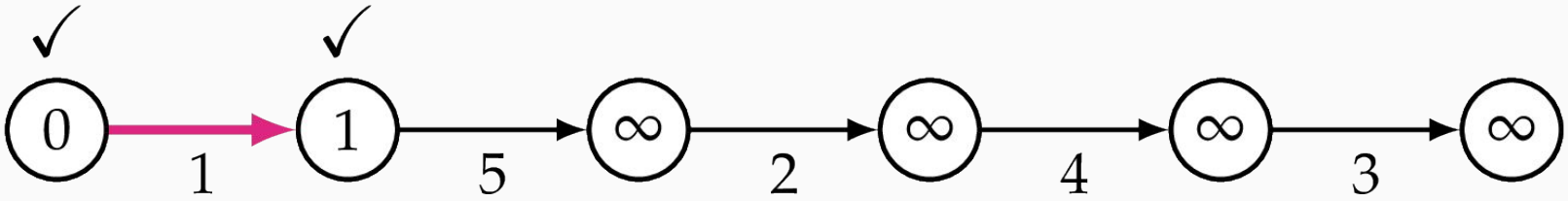
# Relaxing along a path

The labels on vertices along a shortest path will be ***correct*** if the edges are relaxed in ***increasing order of distance*** from the starting vertex at some point in the algorithm:



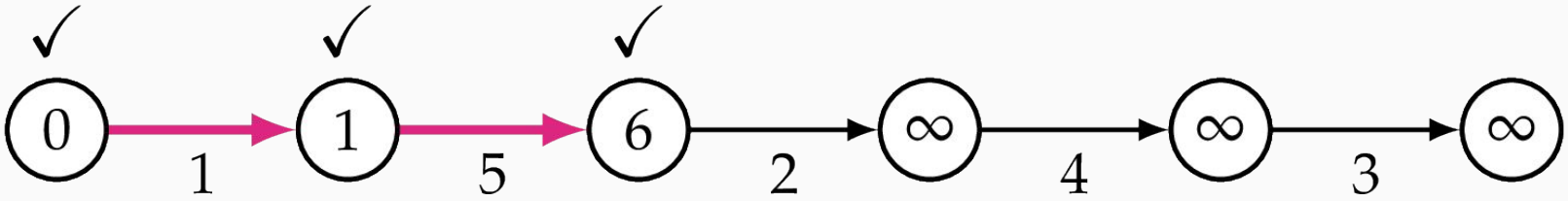
# Relaxing along a path

The labels on vertices along a shortest path will be ***correct*** if the edges are relaxed in ***increasing order of distance*** from the starting vertex at some point in the algorithm:



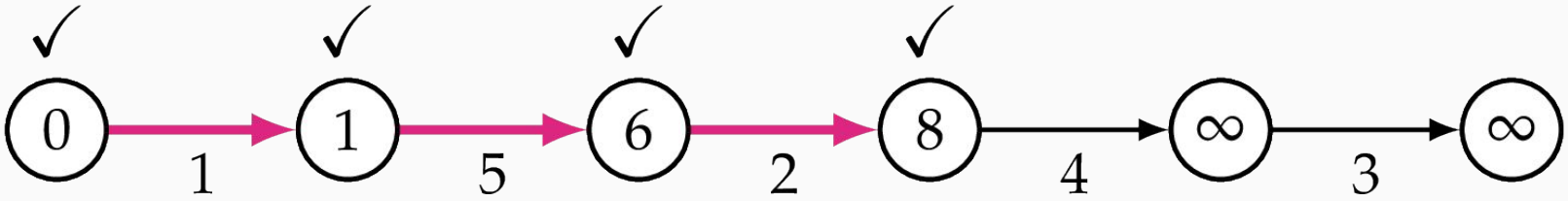
# Relaxing along a path

The labels on vertices along a shortest path will be ***correct*** if the edges are relaxed in ***increasing order of distance*** from the starting vertex at some point in the algorithm:



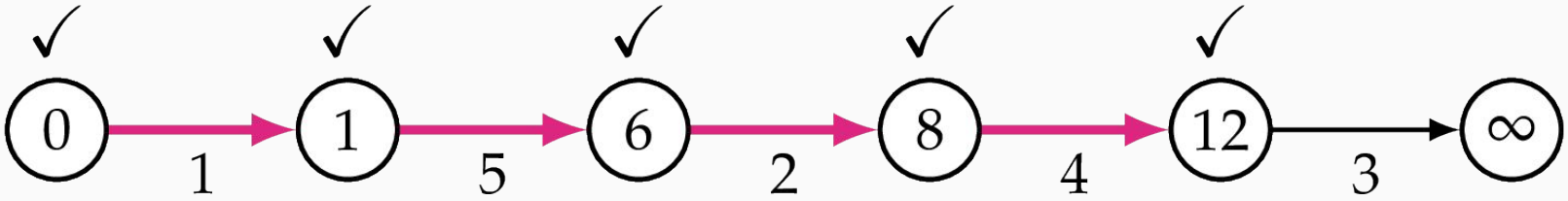
# Relaxing along a path

The labels on vertices along a shortest path will be ***correct*** if the edges are relaxed in ***increasing order of distance*** from the starting vertex at some point in the algorithm:



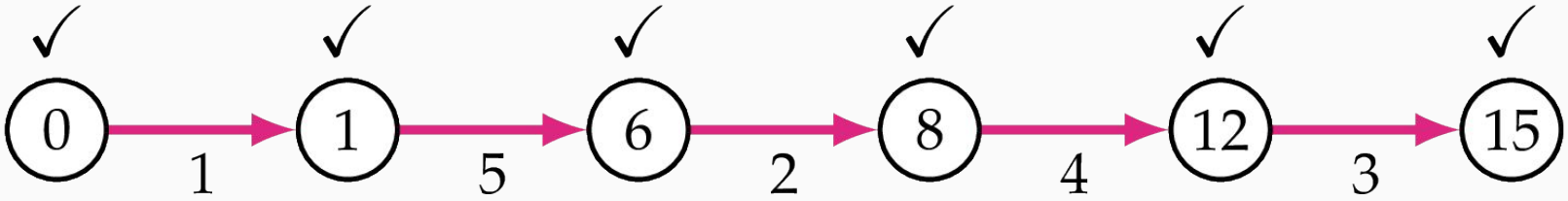
# Relaxing along a path

The labels on vertices along a shortest path will be ***correct*** if the edges are relaxed in ***increasing order of distance*** from the starting vertex at some point in the algorithm:



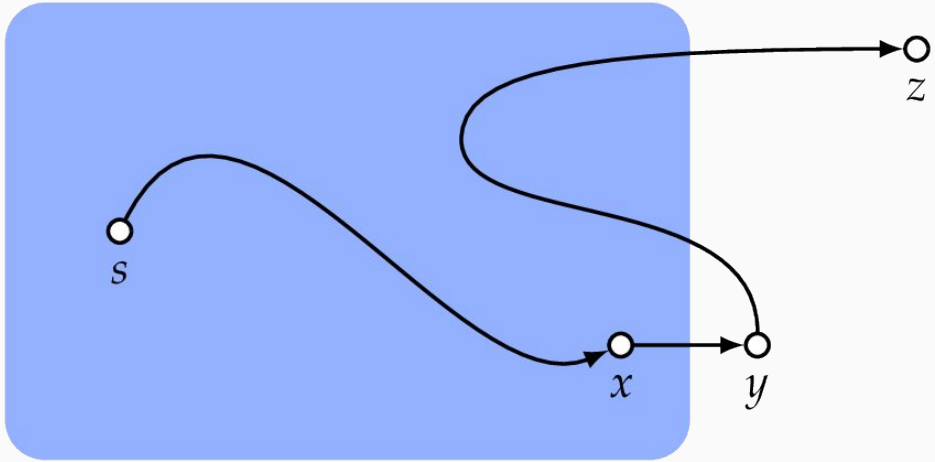
# Relaxing along a path

The labels on vertices along a shortest path will be ***correct*** if the edges are relaxed in ***increasing order of distance*** from the starting vertex at some point in the algorithm:



# Dijkstra’s algorithm - proof

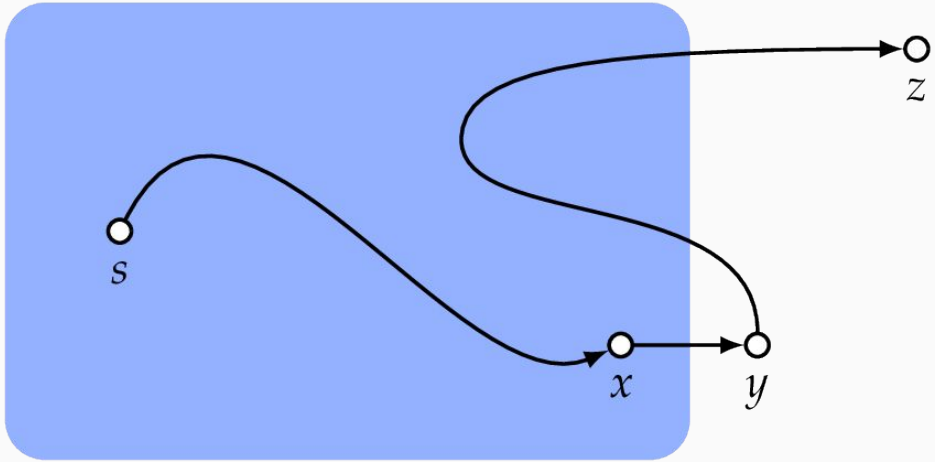
**Induction**: each new vertex ***z*** to the “cloud” has dist(***z***) = D(***s***, ***z***)



# Dijkstra's algorithm - proof

**Induction**: each new vertex ***z*** to the "cloud" has dist(***z***) = D(***s***, ***z***)

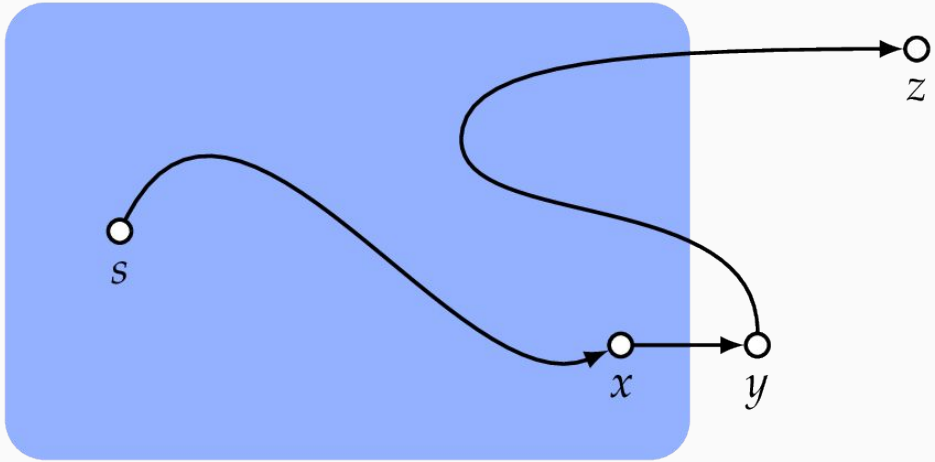
$$
\begin{aligned}
\text{dist}(\boldsymbol{z}) &\le \text{dist}(\boldsymbol{y}) \\
&= D(\boldsymbol{s}, \boldsymbol{y}) \\
&\le D(\boldsymbol{s}, \boldsymbol{y}) + D(\boldsymbol{y}, \boldsymbol{z}) \\
&\le D(\boldsymbol{s}, \boldsymbol{z})
\end{aligned}
$$



# Dijkstra's algorithm - proof

**Induction**: each new vertex ***z*** to the "cloud" has dist(***z***) = D(***s***, ***z***)

$$
\begin{aligned}
\text{dist}(\boldsymbol{z}) &\le \text{dist}(\boldsymbol{y}) \\
&= D(\boldsymbol{s}, \boldsymbol{y}) \\
&\le D(\boldsymbol{s}, \boldsymbol{y}) + D(\boldsymbol{y}, \boldsymbol{z}) \\
&\le D(\boldsymbol{s}, \boldsymbol{z})
\end{aligned}
$$

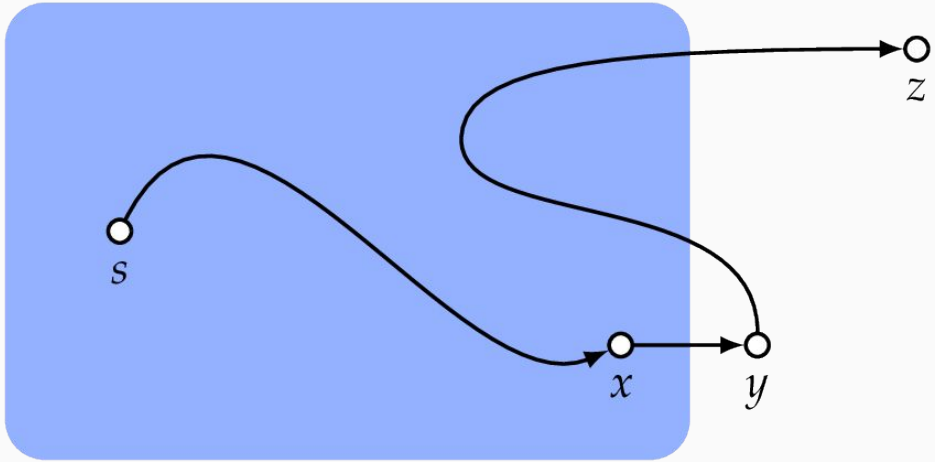



**Argument fails for negative edge weights**

# Dijkstra's algorithm - proof

**Induction**: each new vertex ***z*** to the "cloud" has dist(***z***) = D(***s***, ***z***)

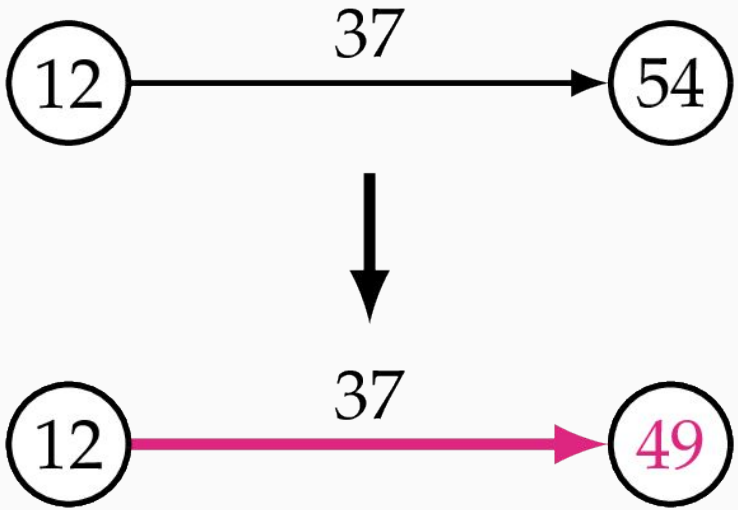
$$
\begin{aligned}
\text{dist}(\boldsymbol{z}) &\le \text{dist}(\boldsymbol{y}) \\
&= D(\boldsymbol{s}, \boldsymbol{y}) \\
&\le D(\boldsymbol{s}, \boldsymbol{y}) + D(\boldsymbol{y}, \boldsymbol{z}) \\
&\le D(\boldsymbol{s}, \boldsymbol{z})
\end{aligned}
$$




**Argument fails for negative edge weights**

# Bellman-Ford

Recall: relax repeatedly until you can't relax



# Bellman-Ford

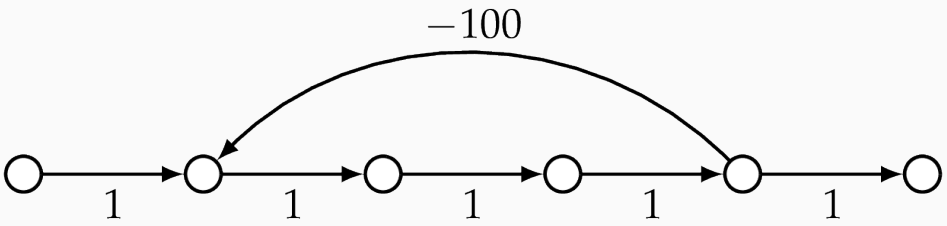
"Relax all the edges… some number of times."

$$
\begin{aligned}
&\textsc{BellmanFord}(G(V,E),s):\\
&\qquad dist(s) := 0\\
&\qquad \textbf{for } v \in V \setminus \{s\}\\
&\qquad\qquad dist(v) := \infty\\
&\qquad \textbf{for } i = 1,\ldots,\boxed{???}\\
&\qquad\qquad \textbf{for } e \in E\\
&\qquad\qquad\qquad \textsc{Relax}(e)
\end{aligned}
$$

# Negative edge weights

SSSP still possible with negative edges, except when there are negative **cycles**:

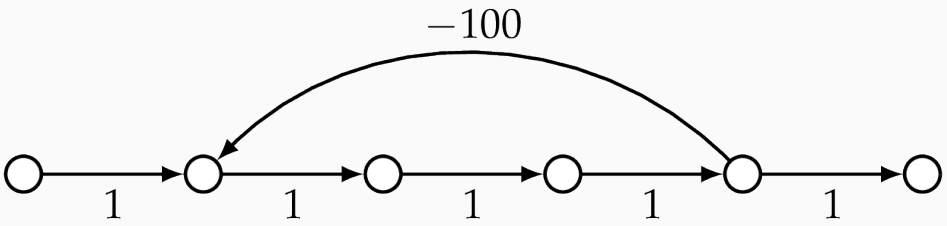
not possible



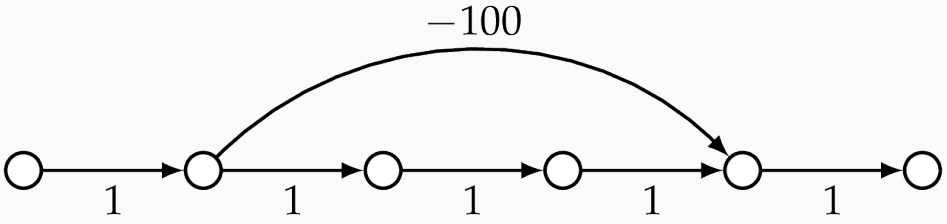
# Negative edge weights

SSSP still possible with negative edges, except when there are negative **cycles**:

not possible



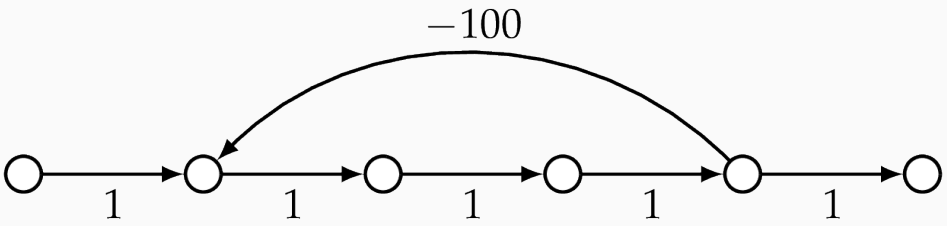

possible



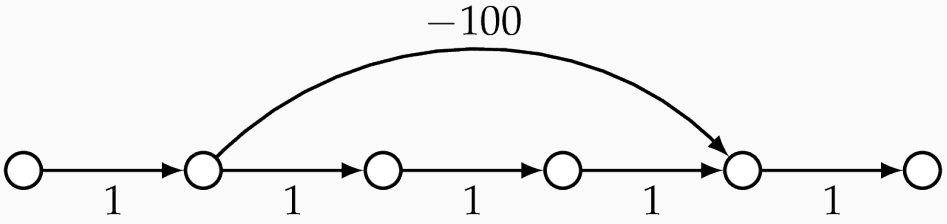
# Negative edge weights

How can we differentiate: **negative cycle** vs. **not done yet**?

not possible




possible



# Bellman-Ford

Claim: after |***V***|-1 iterations, we should be done.

$$
\begin{array}{l}
\textsc{BellmanFord}(G(V,E),s): \\
\quad dist(s) := 0 \\
\quad \textbf{for } v \in V \setminus \{s\} \\
\qquad dist(v) := \infty \\
\quad \textbf{for } i = 1, \ldots, |V| - 1 \\
\qquad \textbf{for } e \in E \\
\qquad\quad \textsc{Relax}(e) \\
\quad \textbf{if } \text{some } e \text{ is tense} \\
\qquad \textbf{return } \textsc{Failed}
\end{array}
$$

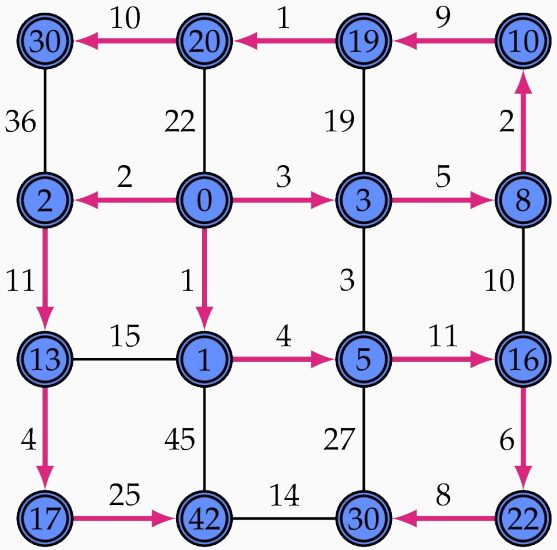
# Bellman-Ford

Claim: after |***V***|-1 iterations, we should be done. Running time: O(|***V***||***E***|)

$$
\begin{aligned}
&\textsc{BellmanFord}(G(V,E),s):\\
&\qquad dist(s) := 0\\
&\qquad \textbf{for } v \in V \setminus \{s\}\\
&\qquad\qquad dist(v) := \infty\\
&\qquad \textbf{for } i = 1,\ldots,|V|-1\\
&\qquad\qquad \textbf{for } e \in E\\
&\qquad\qquad\qquad \textsc{Relax}(e)\\
&\qquad \textbf{if } \text{some } e \text{ is tense}\\
&\qquad\qquad \textbf{return } \textsc{Failed}
\end{aligned}
$$

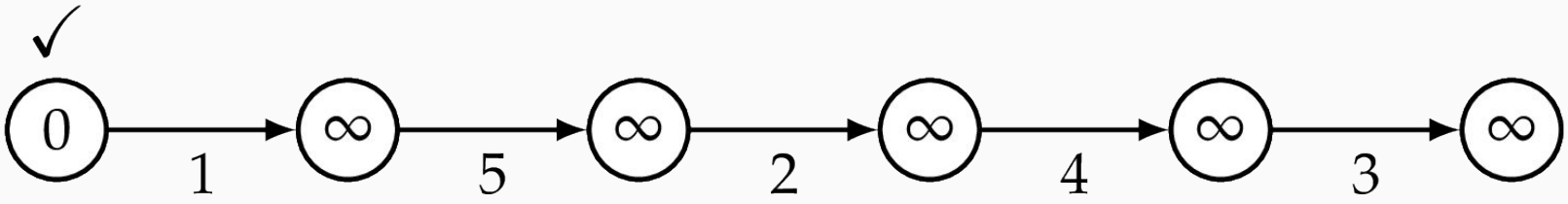
# Bellman-Ford - correctness

Recall: the solution to SSSP is a ***tree*** of shortest paths:



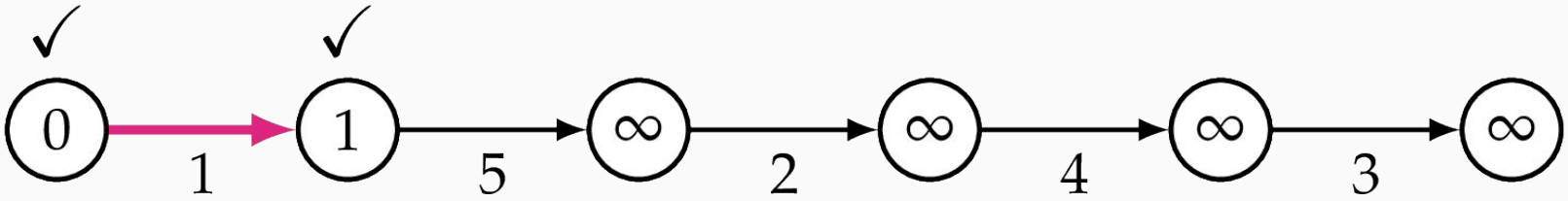
# Bellman-Ford - correctness

Consider any shortest path:



# Bellman-Ford - correctness

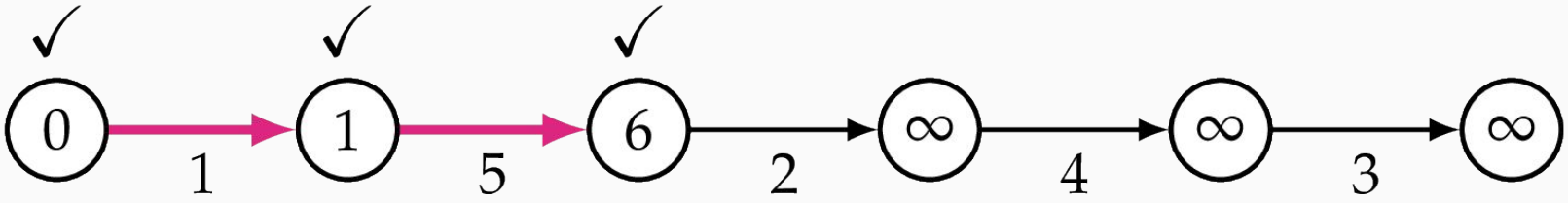
Consider any shortest path:




During first iteration, the first edge is discovered.

# Bellman-Ford - correctness

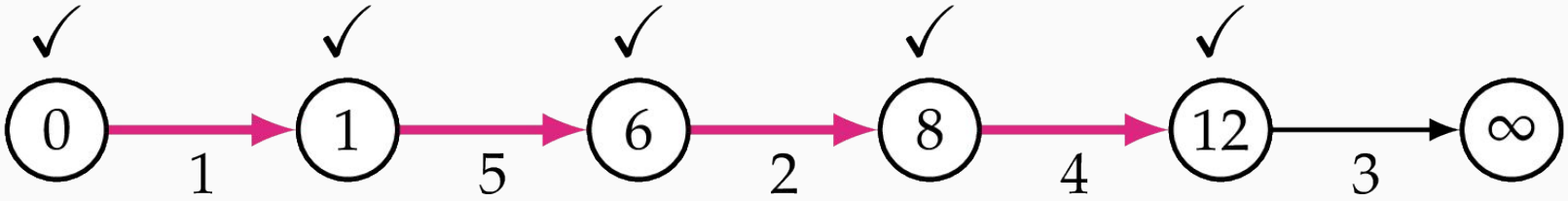
Consider any shortest path:




During second iteration, the second edge is discovered.

# Bellman-Ford - correctness

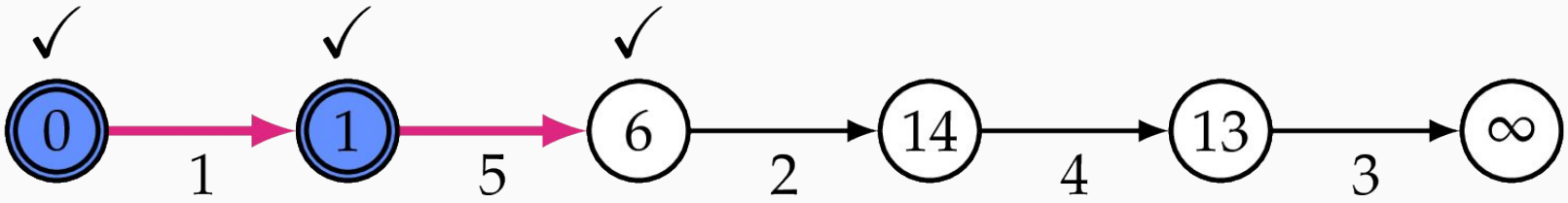
Consider any shortest path:




During *i*th iteration, the *i*th edge is discovered. (Induction, induction, induction…)

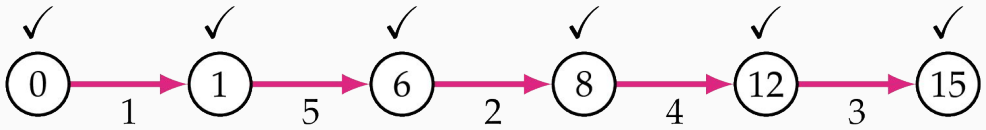
# Dijkstra’s algorithm - alternate proof

If all the edge weights are positive, the vertices along a shortest path have increasing labels and are relaxed in that order:



# Bellman-Ford - correctness

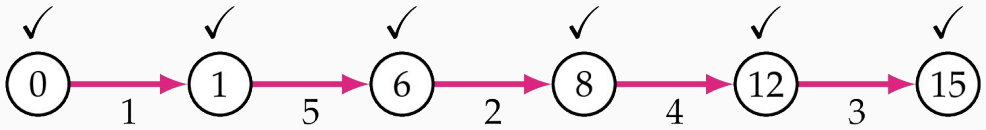
How many edges can be in a shortest path?



# Bellman-Ford - correctness

How many edges can be in a shortest path?

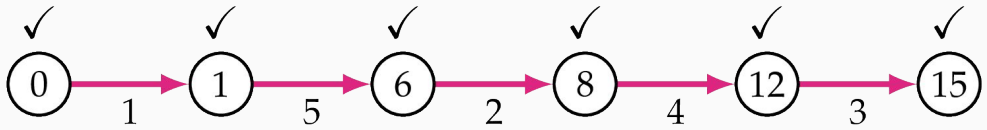
**As many as |*V*|-1** (all the vertices)



# Bellman-Ford - correctness

How many edges can be in a shortest path?

**As many as |*V*|-1** (all the vertices)




$\textsc{BellmanFord}(G(V,E),s)$:

$dist(s) := 0$

**for** $v \in V \setminus \{s\}$

$dist(v) := \infty$

**for** $i = 1, \ldots, \boxed{|V| - 1}$

**for** $e \in E$

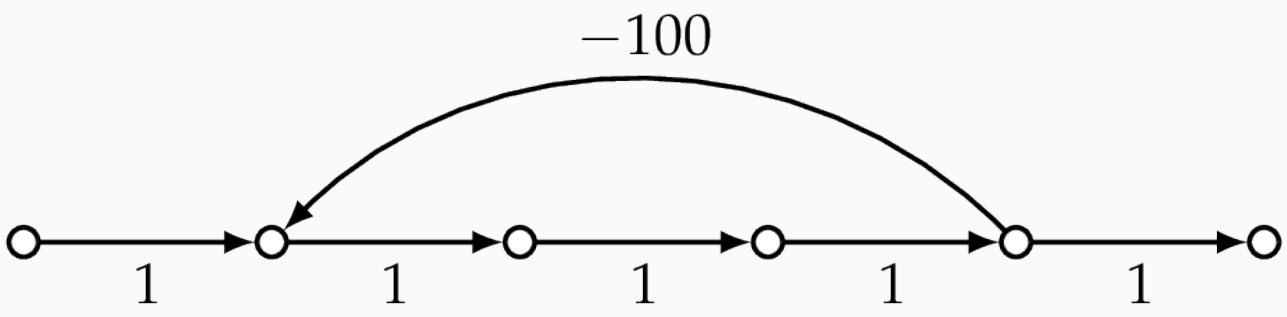
$\textsc{Relax}(e)$

**if** some $e$ is tense

**return** $\textsc{Failed}$

# Bellman-Ford - correctness

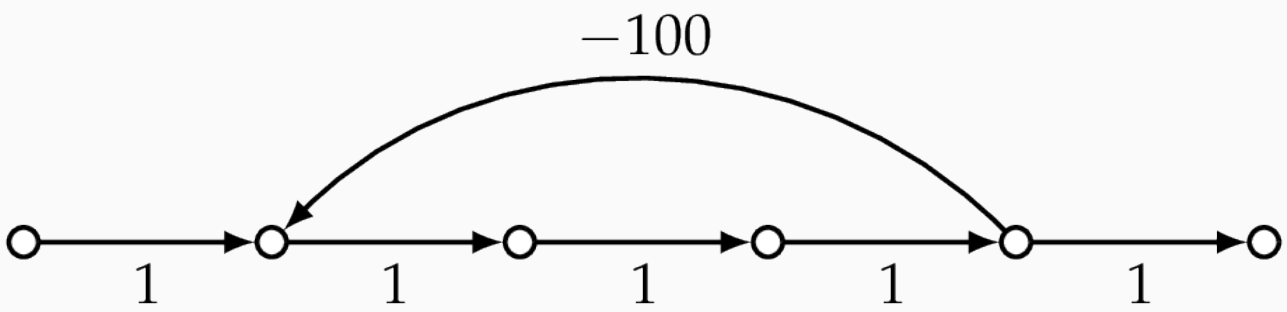
What about negative cycles?



# Bellman-Ford - correctness

What about negative cycles?

**Length of “shortest path” > |*V*|-1 edges**



# Yen’s ordering

A careful edge ordering can lead to a faster algorithm.

# Yen’s ordering

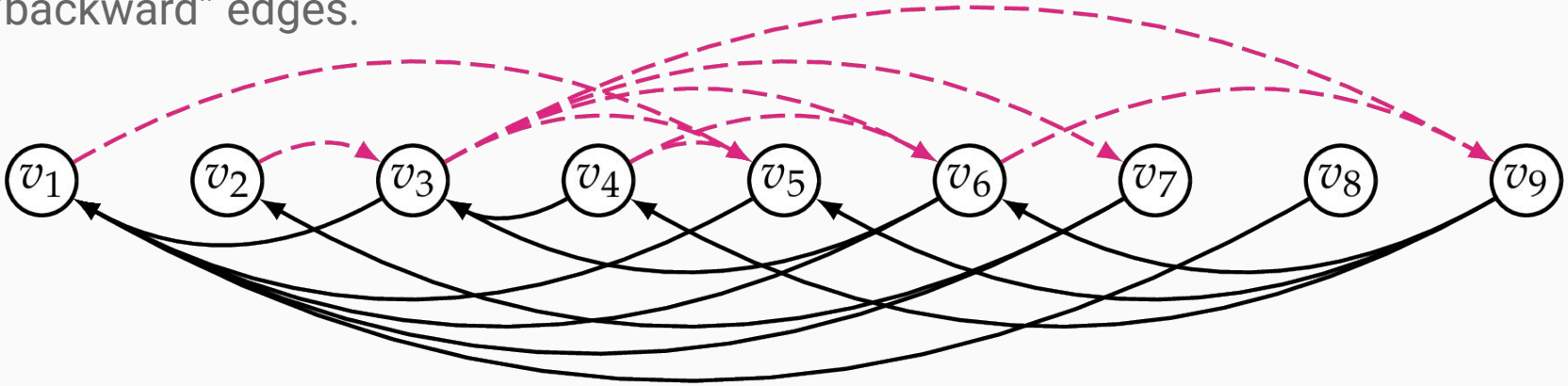
A careful edge ordering can lead to a faster algorithm.

Arbitrarily order the vertices $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n$ and split into “forward” and “backward” edges.

# Yen's ordering

A careful edge ordering can lead to a faster algorithm.

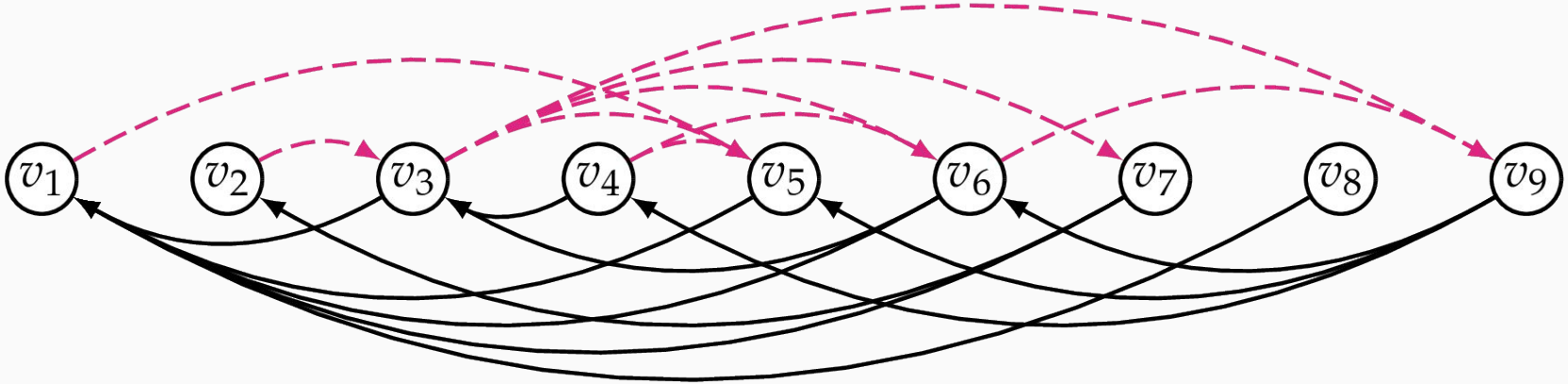
Arbitrarily order the vertices $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n$ and split into "forward" and "backward" edges.



# Yen's ordering

Edge ordering:

- For $\boldsymbol{v}$ in $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n$
  - Relax all red/forward edges leaving $\boldsymbol{v}$.
- For $\boldsymbol{v}$ in $\boldsymbol{v}_n, \ldots, \boldsymbol{v}_2, \boldsymbol{v}_1$
  - Relax all black/back edges leaving $\boldsymbol{v}$.



# Red-then-black paths

If a shortest path were all red edges followed by all black edges:

# Red-then-black paths

If a shortest path were all red edges followed by all black edges:

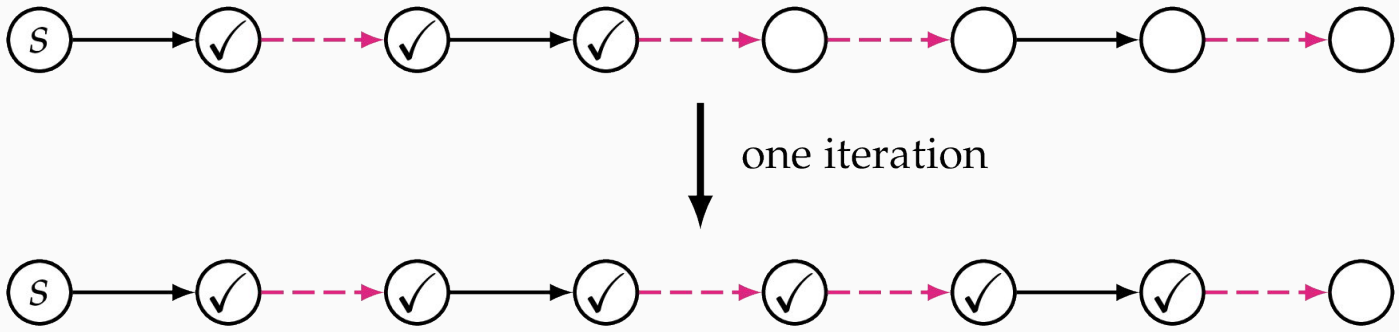
One iteration would be enough

# Red-then-black paths

In general, a shortest path may have many such paths:

# Red-then-black paths

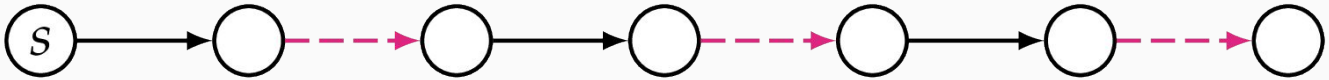
In general, a shortest path may have many such paths:




Each iteration solves one red-then-black subpath.

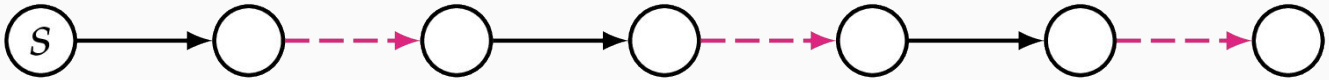
# Red-then-black paths

In the worst case, path alternates black-red-black…

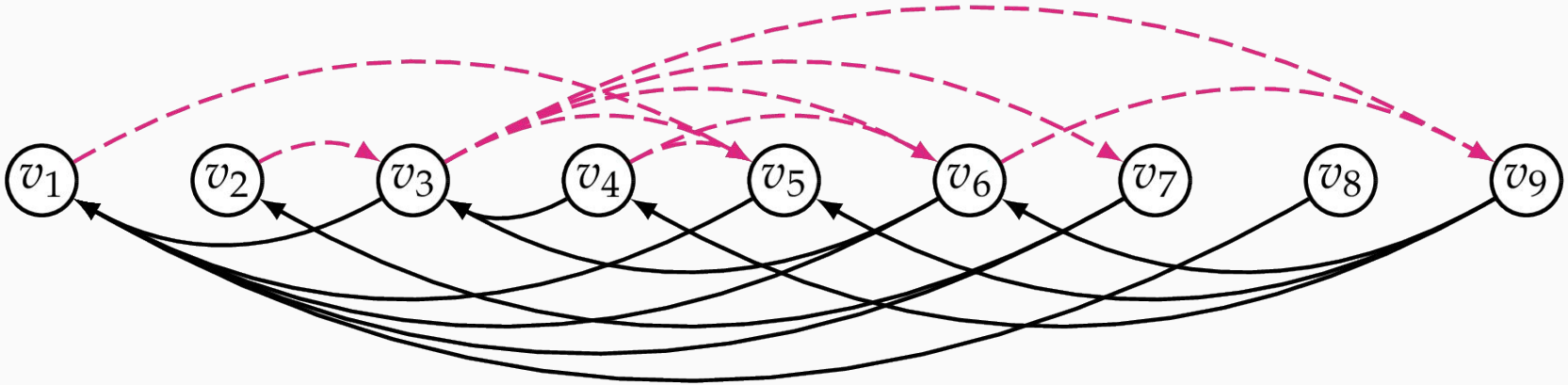


# Red-then-black paths

In the worst case, path alternates black-red-black…




For $|\boldsymbol{V}| = 2\boldsymbol{k}$ or $2\boldsymbol{k}-1$, $\boldsymbol{k}$ paths $\rightarrow \lceil |\boldsymbol{V}|/2 \rceil$ iterations

# Red-then-black paths

Still O(|***V***||***E***|) time, but at least the constant is better.